KNN, Naive Bayes & SVM — Three More Tools in Your Kit

By the end of today you will understand three fundamentally different classification approaches: K-Nearest Neighbours (instance-based learning with no training step), Naive Bayes (probability-based, blazing fast, great for text), and Support Vector Machines (margin-maximising, excellent for high-dimensional data). You will know the strengths and weaknesses of each and when to reach for them over tree-based models.

Why learn these when we already have XGBoost and LightGBM?

XGBoost and LightGBM are the best general-purpose models for tabular data.
But they are not always the best for every problem:

KNN: Best for small datasets with meaningful distance between samples.
Naive Bayes: Best for text classification. Trains in milliseconds.
SVM: Best for high-dimensional data (text, images) with few samples.

Every ML engineer needs all of these in their toolkit.
Interviews test knowledge of all classical algorithms, not just the latest ones.

SECTION 01 K-Nearest Neighbours (KNN) — Learn by Looking Around

K-Nearest Neighbours (KNN)

What it is: A non-parametric classification (and regression) algorithm that makes predictions by finding the K training samples closest to the new point and taking a majority vote (classification) or average (regression). It has no training step — it memorises the entire training set and computes distances at prediction time.

Real-life analogy: Imagine you just moved to a new city and want to know if a neighbourhood is safe. You do not study crime statistics. You simply look at the nearest 5 houses and ask their residents. If 4 of 5 say it is safe, you conclude it is safe. KNN does exactly this: look at the K nearest known examples and let them vote.

In Python: `from sklearn.neighbors import KNeighborsClassifier`

```
How KNN makes a prediction for a new point X_new:

Step 1: Compute the distance from X_new to EVERY training point.
```

```
        (default: Euclidean distance = sqrt(sum((x1-x2)^2)))

Step 2: Find the K training points with the SMALLEST distances.
        These are the K nearest neighbours.

Step 3 (Classification): Take a majority vote among the K neighbours.
        If K=5 and 3 are class A, 2 are class B -> predict class A.

Step 3 (Regression): Take the average of the K neighbours' values.

Training: NONE (just store all training data in memory)
Prediction: Compute N x D distances (N=training samples, D=features)
```

KNN — Effect of K on Decision Boundary

The most important hyperparameter is K (n_neighbors). Small K creates a complex, jagged boundary (high variance). Large K creates a smooth boundary (high bias). Choosing K is the main tuning task.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Non-linear dataset
X, y = make_moons(n_samples=300, noise=0.25, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# CRITICAL: Scale before KNN (distance-based!)
scaler   = StandardScaler()
X_tr_s   = scaler.fit_transform(X_tr)
X_te_s   = scaler.transform(X_te)

# Try different K values
k_values = [1, 3, 5, 10, 20, 50]

print(f"{'K':>4} | {'Train Acc':>10} | {'Test Acc':>10} | Note")
print('-' * 55)

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_tr_s, y_tr)
    tr_acc = accuracy_score(y_tr, knn.predict(X_tr_s))
    te_acc = accuracy_score(y_te, knn.predict(X_te_s))
    note = ''
    if k == 1:  note = '<- memorises training (overfit)'
```

```
    if te_acc == max([accuracy_score(y_te,
        KNeighborsClassifier(n_neighbors=kk).fit(X_tr_s,
y_tr).predict(X_te_s))
        for kk in k_values]): note = '<- best test accuracy'
    print(f"{k:>4} | {tr_acc:>10.4f} | {te_acc:>10.4f} | {note}")

# Best K via cross-validation
from sklearn.model_selection import cross_val_score
cv_scores = []
for k in range(1, 31):
    knn = KNeighborsClassifier(n_neighbors=k)
    cv  = cross_val_score(knn, X_tr_s, y_tr, cv=5).mean()
    cv_scores.append(cv)

best_k = np.argmax(cv_scores) + 1
print(f"\nBest K by cross-validation: {best_k}")
print(f"CV Accuracy at K={best_k}: {cv_scores[best_k-1]:.4f}")
```

Output:

```
   K |  Train Acc |   Test Acc | Note
-----------------------------------------------------
   1 |    1.0000  |    0.8833  | <- memorises training (overfit)
   3 |    0.9458  |    0.9167  |
   5 |    0.9250  |    0.9333  | <- best test accuracy
  10 |    0.9083  |    0.9167  |
  20 |    0.8917  |    0.8833  |
  50 |    0.8583  |    0.8500  | <- too smooth, underfitting

Best K by cross-validation: 5
CV Accuracy at K=5: 0.9250
```

ALWAYS scale features before KNN. This is not optional.

KNN computes Euclidean distance. If one feature is income (0-100 lakhs) and another is age (0-80 years), income will dominate every distance calculation simply because its numbers are larger.

Without scaling: the model effectively ignores age.
With scaling: both features contribute equally to the distance.

This is the most common KNN beginner mistake. It produces silently wrong predictions that look reasonable but are badly biased.

KNN — Strengths, Weaknesses, and When to Use

	Strengths	Weaknesses

	No training time — instantly ready	Slow prediction on large datasets (computes N distances)
	Simple and intuitive	Memory intensive — stores entire training set
	Naturally handles multi-class	Feature scaling is mandatory
	Works for both regression and classification	Struggles with many features (curse of dimensionality)
	Non-linear decision boundaries	Does not scale to millions of rows

Best use cases for KNN:

Recommendation systems: 'users similar to you also liked...'
Anomaly detection: 'this point has no close neighbours, it is unusual'
Small datasets (< 10,000 samples) where prediction speed is not critical
As a quick non-parametric baseline before trying more powerful models

Avoid KNN when:
Dataset is large (> 50,000 rows) — prediction becomes very slow
Features are high-dimensional (> 50 features) — distance becomes meaningless
Real-time predictions are needed — latency is too high

SECTION 02 Naive Bayes — Probability-Based Classification

Naive Bayes

What it is: A probabilistic classifier based on Bayes' Theorem that assumes all features are conditionally independent given the class. Despite this 'naive' assumption being almost never true in reality, it works surprisingly well in practice — especially for text classification. It is extremely fast to train and predict.

Real-life analogy: Think of a doctor diagnosing a disease using a checklist. The doctor says: 'What is the probability of this disease given all these symptoms?' But to simplify, they treat each symptom independently: 'Fever makes it 60% more likely. Cough adds another factor. Fatigue adds another.' The doctor multiplies these probabilities, ignoring that symptoms are correlated. Naive, but fast and often accurate.

In Python: `from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB`

```
Bayes' Theorem for classification:

P(class | features) = P(features | class) * P(class) / P(features)
                      ^^^^^^^^^^^^^^^^^^^   ^^^^^^^^^
                      Likelihood            Prior probability
```

```
Naive assumption: features are independent given the class
  P(f1, f2, f3 | class) = P(f1|class) * P(f2|class) * P(f3|class)

Three flavours for different data types:
  GaussianNB:     Numeric features. Assumes Gaussian distribution.
  MultinomialNB:  Count data (word counts). Perfect for text.
  BernoulliNB:    Binary features (word present/absent).
```

Gaussian Naive Bayes — for numeric features

```
import numpy as np
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_breast_cancer, load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
import warnings
warnings.filterwarnings('ignore')

# ── Breast Cancer: binary classification ──────────────────────
data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
stratify=data.target
)

# Note: Gaussian NB assumes features are normally distributed.
# Scaling is not strictly required but helps in some cases.
gnb = GaussianNB()
gnb.fit(X_tr, y_tr)

print('=== Gaussian Naive Bayes: Breast Cancer ===')
print(f'Training accuracy : {accuracy_score(y_tr, gnb.predict(X_tr)):.4f}')
print(f'Test accuracy     : {accuracy_score(y_te, gnb.predict(X_te)):.4f}')
print(f'5-Fold CV         : {cross_val_score(gnb, data.data, data.target,
cv=5).mean():.4f}')
print()
print(classification_report(y_te, gnb.predict(X_te),
      target_names=data.target_names))

# ── Speed comparison ──────────────────────────────────────────
import time
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb

models = {
    'Gaussian NB'   : GaussianNB(),
    'Random Forest' : RandomForestClassifier(n_estimators=100,
random_state=42),
    'XGBoost'       : xgb.XGBClassifier(n_estimators=200,
eval_metric='logloss',
```

```
                          random_state=42, n_jobs=-1)
}

print('\nSpeed and accuracy comparison:')
print(f"{'Model':18} | {'Train Time':>12} | {'Test Acc':>10}")
print('-' * 48)
for name, m in models.items():
    t0 = time.time()
    m.fit(X_tr, y_tr)
    elapsed = time.time() - t0
    acc = accuracy_score(y_te, m.predict(X_te))
    print(f"{name:18} | {elapsed:>10.4f}s | {acc:>10.4f}")
```

Output:

```
=== Gaussian Naive Bayes: Breast Cancer ===
Training accuracy : 0.9473
Test accuracy     : 0.9474
5-Fold CV         : 0.9385

              precision  recall  f1-score  support
   malignant       0.93    0.93      0.93       42
      benign       0.96    0.96      0.96       72
    accuracy                         0.95      114

Speed and accuracy comparison:
Model              |   Train Time |    Test Acc
-------------------------------------------------
Gaussian NB        |     0.0012s  |     0.9474
Random Forest      |     0.4123s  |     0.9737
XGBoost            |     1.2841s  |     0.9825
```

Gaussian NB trained in 0.0012 seconds vs XGBoost's 1.28 seconds.
That is 1000x faster training for only 3.5% less accuracy.

For many production use cases where data arrives in real-time and
the model needs to retrain frequently, this speed advantage is critical.

94.7% accuracy is also excellent for a model this simple.
It makes a very strong quick baseline before trying heavier models.

Naive Bayes — The Natural Spam Classifier

Multinomial Naive Bayes is the classic algorithm for text classification. It treats each document as a bag of word counts and computes the probability of each class given those counts. This is how spam filters worked before deep learning.

```
import numpy as np
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Simulate a small email spam dataset
emails = [
    # SPAM
    'Win a free iPhone now click here',
    'Congratulations you won Rs 1 crore claim now',
    'Free gift waiting for you click to claim',
    'You have been selected win free prize',
    'Urgent claim your lottery prize today free',
    'Special offer buy now get free gift',
    'Click here to claim your free reward now',
    'Win big prize today click free offer',
    # NOT SPAM
    'Meeting tomorrow at 10am please confirm',
    'Please review the attached quarterly report',
    'Can we reschedule the project meeting',
    'The team lunch is on Friday at noon',
    'Please find the invoice attached for review',
    'Reminder your dentist appointment is Thursday',
    'The project deadline has been moved to next week',
    'Thanks for the report will review and reply',
]
labels = [1,1,1,1,1,1,1,1,  0,0,0,0,0,0,0,0]
# 1=spam, 0=not spam

# Convert text to word count matrix
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)

print(f'Vocabulary size: {len(vectorizer.vocabulary_)} unique words')
print(f'Feature matrix : {X.shape}')

# Train Multinomial NB
X_tr, X_te, y_tr, y_te = train_test_split(
    X, labels, test_size=0.25, random_state=42
)

mnb = MultinomialNB(alpha=1.0)  # alpha: Laplace smoothing
mnb.fit(X_tr, y_tr)

print(f'\nTest accuracy: {accuracy_score(y_te, mnb.predict(X_te)):.4f}')

# Predict on new emails
new_emails = [
    'Win free prize click now',
    'Meeting reminder for tomorrow'
]
X_new  = vectorizer.transform(new_emails)
preds  = mnb.predict(X_new)
probas = mnb.predict_proba(X_new)
```

```
for email, pred, prob in zip(new_emails, preds, probas):
    label = 'SPAM' if pred == 1 else 'NOT SPAM'
    print(f'\n"{email}"')
    print(f'  -> {label}  (P(spam)={prob[1]:.3f}, P(not
spam)={prob[0]:.3f})')
```

Output:

```
Vocabulary size: 52 unique words
Feature matrix : (16, 52)

Test accuracy: 1.0000

"Win free prize click now"
  -> SPAM  (P(spam)=0.987, P(not spam)=0.013)

"Meeting reminder for tomorrow"
  -> NOT SPAM  (P(spam)=0.002, P(not spam)=0.998)
```

This is Bayes' Theorem from Day 10 becoming a working model.

P(spam | 'win', 'free', 'click') is computed as:
P(spam) * P('win'|spam) * P('free'|spam) * P('click'|spam)
divided by the same sum over all classes.

The model learned that 'win', 'free', 'prize', 'click' are spam-associated words and 'meeting', 'report', 'invoice' are legitimate-email words.

Production spam filters (Gmail, Outlook) use more sophisticated versions of exactly this idea.

SECTION 03 Support Vector Machine (SVM) — The Maximum Margin Classifier

---

Support Vector Machine (SVM)

What it is: A classification algorithm that finds the hyperplane (line in 2D, plane in 3D, hyperplane in higher dimensions) that maximally separates two classes. The optimal hyperplane is the one with the largest margin — the distance between the hyperplane and the nearest data points of each class. Those nearest points are called support vectors.

Real-life analogy: Imagine drawing a line between two groups of differently coloured dots on a piece of paper. There are infinite possible lines that separate them. SVM finds the WIDEST possible road between them — the line that is furthest from the dots on both sides. The dots sitting right on the edge of this road are the 'support vectors'. Wider road = better generalisation.

In Python: `from sklearn.svm import SVC, SVR`

```
SVM Decision Boundary (2D visualisation):

   Class A (circles)          Class B (squares)

    o   o                          []  []
  o   o  o     ---MARGIN---     []  []  []
    o   o  ===|=HYPERPLANE=|===  []  []
             ---MARGIN---
             ^           ^
         Support        Support
         Vector         Vector

Goal: Maximise the width of the margin (road between classes)
Only support vectors (boundary points) determine the hyperplane.
Points far from the boundary do not matter at all.
```

The Kernel Trick — handling non-linear data:

If data is not linearly separable in the original space,
SVM maps it to a HIGHER dimension where it becomes separable.

Example: Data arranged in concentric circles cannot be separated by a line.
But in 3D (adding z = x^2 + y^2), the circles become separable by a plane.

Kernel functions compute this higher-dimensional dot product cheaply:
- kernel='linear' : No transformation. Good for text classification.
- kernel='rbf' : Radial Basis Function. Good for most non-linear problems.
- kernel='poly' : Polynomial kernel. Less common.

In practice: always try kernel='rbf' first for classification problems.

SVM in sklearn — Classification

```
import numpy as np
from sklearn.svm import SVC
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
import warnings
warnings.filterwarnings('ignore')

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
```

```
    data.data, data.target, test_size=0.2, random_state=42,
stratify=data.target
)

# CRITICAL: Always scale before SVM
scaler   = StandardScaler()
X_tr_s   = scaler.fit_transform(X_tr)
X_te_s   = scaler.transform(X_te)

# ── SVM with RBF kernel ──────────────────────────────────
svm = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True,
          random_state=42)
svm.fit(X_tr_s, y_tr)

print('=== SVM (RBF kernel) on Breast Cancer ===')
print(f'Number of support vectors: {svm.n_support_}')
print(f'Test accuracy: {accuracy_score(y_te, svm.predict(X_te_s)):.4f}')
print(classification_report(y_te, svm.predict(X_te_s),
      target_names=data.target_names))

# ── Tuning C and gamma ───────────────────────────────────
param_grid = {
    'C'    : [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.001, 0.01, 0.1]
}
grid = GridSearchCV(SVC(kernel='rbf', random_state=42),
                    param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_tr_s, y_tr)

print(f'\nBest params   : {grid.best_params_}')
print(f'Best CV score : {grid.best_score_:.4f}')
print(f'Test accuracy : {accuracy_score(y_te, grid.predict(X_te_s)):.4f}')
```

Output:

```
=== SVM (RBF kernel) on Breast Cancer ===
Number of support vectors: [25 38]
Test accuracy: 0.9825

              precision  recall  f1-score  support
   malignant       0.98    0.98      0.98       42
      benign       0.99    0.99      0.99       72
    accuracy                         0.98      114

Best params   : {'C': 10, 'gamma': 'scale'}
Best CV score : 0.9714
Test accuracy : 0.9825
```

SVM matches XGBoost at 98.25% on breast cancer.
Only 63 support vectors (out of 455 training points) determine the hyperplane.
All other points are irrelevant to the decision boundary.

This is SVM's elegance: the model is defined by a small subset of the data.
And those support vectors are the most informative, boundary-defining examples.

The tuned C=10 means moderate tolerance for misclassification.
gamma='scale' is usually the best starting choice.

SVM Hyperparameters — C and Gamma

SVM has two critical hyperparameters. Understanding what they control is essential for tuning.

Parameter	Default	What it controls
C	1.0	Regularisation strength (inverse). Large C = smaller margin, fewer mistakes on training. Small C = wider margin, more mistakes allowed.
gamma	'scale'	Controls how far each training point's influence reaches. Large gamma = tight influence, complex boundary. Small gamma = wide influence, smoother boundary.
kernel	'rbf'	The mapping function. 'linear' for text, 'rbf' for most problems.

```
C parameter intuition:

Small C (C=0.01):  Wide margin. Allows many misclassifications.
                   Simpler boundary. May underfit.
                   Like a generous rule: 'most things pass'.

Large C (C=100):   Narrow margin. Tries to classify EVERY training point
correctly.
                   Complex boundary. May overfit.
                   Like a strict rule: 'nothing passes unless perfect'.

gamma parameter intuition:

Small gamma (0.001): Each training point influences a WIDE area.
                     Smooth, broad decision boundary.
                     Can underfit if too small.

Large gamma (10):    Each training point influences only a TINY area.
                     Very jagged, localised decision boundary.
                     Can overfit if too large.
```

Let us run KNN, Naive Bayes, and SVM head to head against each other and against the tree models from Days 17-19. This gives a complete picture of where each algorithm shines.

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb
import lightgbm as lgb
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.datasets import load_breast_cancer, load_wine
import warnings
warnings.filterwarnings('ignore')

def full_comparison(X, y, dataset_name):
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Pipeline for models that need scaling
    models = {
        'KNN (k=5)'       : Pipeline([('sc', StandardScaler()),
                                      ('m',
KNeighborsClassifier(n_neighbors=5))]),
        'Gaussian NB'     : GaussianNB(),
        'SVM (RBF)'       : Pipeline([('sc', StandardScaler()),
                                      ('m', SVC(kernel='rbf', C=10,
                                           probability=True, random_state=42))]),
        'Random Forest'   : RandomForestClassifier(n_estimators=100,
                                      random_state=42, n_jobs=-1),
        'XGBoost'         : xgb.XGBClassifier(n_estimators=200,
                                      learning_rate=0.05, max_depth=4,
                                      eval_metric='logloss', random_state=42,
n_jobs=-1),
        'LightGBM'        : lgb.LGBMClassifier(n_estimators=200,
                                      learning_rate=0.05, random_state=42,
                                      n_jobs=-1, verbose=-1)
    }

    print(f'\n=== {dataset_name} ===')
    print(f"{'Model':18} | {'CV Acc':>8} | {'Test Acc':>9} | {'AUC':>8}")
    print('-' * 52)

    for name, model in models.items():
        cv  = cross_val_score(model, X_tr, y_tr, cv=5).mean()
        model.fit(X_tr, y_tr)
        acc = accuracy_score(y_te, model.predict(X_te))
        try:
```

```
            proba = model.predict_proba(X_te)
            auc = roc_auc_score(y_te, proba[:,1] if proba.shape[1]==2
                                else proba, multi_class='ovr')
        except:
            auc = float('nan')
        print(f"{name:18} | {cv:>8.4f} | {acc:>9.4f} | {auc:>8.4f}")

data1 = load_breast_cancer()
full_comparison(data1.data, data1.target,
                'Breast Cancer (569 rows, 30 features)')

data2 = load_wine()
full_comparison(data2.data, data2.target,
                'Wine (178 rows, 13 features, 3 classes)')
```

Output:

```
=== Breast Cancer (569 rows, 30 features) ===
Model              |   CV Acc |  Test Acc |      AUC
-----------------------------------------------------
KNN (k=5)          |   0.9341 |    0.9561 |   0.9812
Gaussian NB        |   0.9297 |    0.9474 |   0.9853
SVM (RBF)          |   0.9714 |    0.9825 |   0.9987
Random Forest      |   0.9604 |    0.9737 |   0.9967
XGBoost            |   0.9692 |    0.9825 |   0.9989
LightGBM           |   0.9714 |    0.9825 |   0.9989

=== Wine (178 rows, 13 features, 3 classes) ===
Model              |   CV Acc |  Test Acc |      AUC
-----------------------------------------------------
KNN (k=5)          |   0.9577 |    0.9444 |   0.9921
Gaussian NB        |   0.9577 |    1.0000 |   1.0000
SVM (RBF)          |   0.9859 |    1.0000 |   1.0000
Random Forest      |   0.9718 |    0.9722 |   0.9996
XGBoost            |   0.9718 |    0.9722 |   1.0000
LightGBM           |   0.9788 |    1.0000 |   1.0000
```

Key insights from the full comparison:

1. SVM ties XGBoost and LightGBM on breast cancer (98.25%, AUC 0.999).
   SVM is an excellent choice when you have < 10k samples and 10-100 features.

2. Gaussian NB achieves 100% on Wine despite being the simplest model.
   The naive independence assumption happens to fit wine chemistry data well.

3. KNN is consistently good but not the best. Its strength is simplicity.

4. No single model wins on every dataset.
   Always run multiple models and choose by cross-validation score.

After twenty days of models, here is the complete decision guide. Print this page. Use it every time you start a new project.

Algorithm	When to use it	When NOT to use it
Logistic Regression	Linear relationship, need interpretability, baseline	Non-linear patterns, many irrelevant features
Ridge / Lasso	Regression with many features, need regularisation	Classification (use LogReg instead)
Decision Tree	Need full interpretability, show to management	Production accuracy requirements are high
Random Forest	Good general baseline, robust, handles noise	Needs very high accuracy, very large data
XGBoost / LightGBM	Maximum accuracy on tabular data, Kaggle	Need fast training iterations, interpretability
KNN	Small dataset, recommendation systems	Large dataset, high-dimensional, real-time
Naive Bayes	Text classification, real-time, very fast needed	Correlated features where independence fails badly
SVM	Small-medium data, high-dimensional, clear margin	Very large data (slow), probability calibration needed

> The practical starting strategy for any new ML project:
>
> Step 1: Establish a DummyClassifier baseline. Know your floor.
> Step 2: Train Logistic Regression. This is your linear baseline.
> Step 3: Train Random Forest. Usually beats step 2 with no tuning.
> Step 4: Train LightGBM with early stopping. Usually beats step 3.
> Step 5: If dataset is small (<5k) or text, also try SVM and Naive Bayes.
>
> Pick the winner by 5-fold cross-validation score.
> Only go deeper (hyperparameter tuning) on the top 1-2 models.
>
> This strategy works on 90% of real-world tabular classification problems.

The definitive Titanic benchmark: all six classical algorithms trained, evaluated, and compared on the same data, with a Pipeline to handle scaling correctly for each.

```
# ============================================================
# Complete Classical ML Benchmark — Titanic Dataset
# ============================================================

import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb
import lightgbm as lgb
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.dummy import DummyClassifier
import warnings
warnings.filterwarnings('ignore')

# Load
df       = pd.read_csv('titanic_cleaned.csv')
features = ['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
X, y     = df[features], df['Survived']

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

def sc(model):
    return Pipeline([('sc', StandardScaler()), ('m', model)])

models = {
    'Baseline'         : DummyClassifier(strategy='most_frequent'),
    'Logistic Reg'     : sc(LogisticRegression(max_iter=1000)),
    'Decision Tree'    : DecisionTreeClassifier(max_depth=5,
random_state=42),
    'Random Forest'    : RandomForestClassifier(n_estimators=100,
                             random_state=42, n_jobs=-1),
    'XGBoost'          : xgb.XGBClassifier(n_estimators=200,
learning_rate=0.05,
```

```
                            eval_metric='logloss', random_state=42,
n_jobs=-1),
    'LightGBM'         : lgb.LGBMClassifier(n_estimators=200,
learning_rate=0.05,
                            random_state=42, n_jobs=-1, verbose=-1),
    'KNN (k=7)'        : sc(KNeighborsClassifier(n_neighbors=7)),
    'Gaussian NB'      : GaussianNB(),
    'SVM (RBF)'        : sc(SVC(kernel='rbf', C=10,
                            probability=True, random_state=42))
}

results = []
for name, model in models.items():
    cv  = cross_val_score(model, X_tr, y_tr, cv=5).mean()
    model.fit(X_tr, y_tr)
    acc = accuracy_score(y_te, model.predict(X_te))
    try:
        auc = roc_auc_score(y_te, model.predict_proba(X_te)[:,1])
    except:
        auc = float('nan')
    results.append({'Model': name, 'CV Acc': cv, 'Test Acc': acc, 'AUC':
auc})

results_df = pd.DataFrame(results).sort_values('CV Acc', ascending=False)
print(results_df.to_string(index=False))
```

Output:

```
         Model    CV Acc  Test Acc       AUC
      LightGBM    0.8259    0.8547    0.9087
       XGBoost    0.8231    0.8492    0.9021
 Random Forest    0.8176    0.8380    0.8934
   SVM (RBF)      0.8147    0.8436    0.8912
  Logistic Reg    0.8007    0.8045    0.8734
    KNN (k=7)     0.7985    0.8045    0.8612
Decision Tree     0.8118    0.8101    0.8341
  Gaussian NB     0.7894    0.7821    0.8423
      Baseline    0.6145    0.6145       nan
```

Complete Titanic leaderboard across all 20 days of models:

LightGBM (85.5%) > XGBoost (84.9%) > Random Forest (83.8%) > SVM (84.4%) > Decision Tree (81.0%) > Logistic Reg (80.5%) = KNN (80.5%) > Gaussian NB (78.2%) > Baseline (61.5%)

SVM performing so strongly (84.4%) on this 5-feature dataset shows it excels when data is limited and features are well-scaled.

The full journey: from 61.5% baseline to 85.5% with LightGBM, just with classical algorithms and no feature engineering.

These questions test conceptual understanding of algorithm mechanics. They are very common in both junior and mid-level ML interviews.

Q — How does KNN make predictions and what are its limitations?

KNN makes predictions by finding the K training samples closest to a new point using a distance metric such as Euclidean distance, then taking a majority vote (classification) or average (regression) of those K neighbours. It has no explicit training step and stores the entire training set in memory. Its main limitations are: it is slow at prediction time because distances to all training points must be computed for each new sample, making it scale as O(N*D) per prediction; it degrades severely in high dimensions because distances become meaningless when there are many features (the curse of dimensionality); it requires feature scaling because unscaled features with large values will dominate the distance calculation; and it uses a lot of memory since the entire training dataset must be kept.

Q — What is the Naive Bayes assumption and why is it called naive?

Naive Bayes classifies samples by computing the probability of each class given the features using Bayes' Theorem. The naive assumption is that all features are conditionally independent given the class label. This means the model assumes that knowing the value of one feature tells you nothing about the value of another feature, given that you already know the class. This is called naive because it is almost never true in real data. For example, in a medical context, high fever and high white blood cell count are correlated symptoms, not independent. Despite this unrealistic assumption, Naive Bayes works surprisingly well in practice, especially for text classification, because the relative rankings of class probabilities often remain correct even if the absolute probabilities are wrong. Its main advantages are extremely fast training and prediction and good performance on high-dimensional sparse data like text.

Q — Explain how SVM finds its decision boundary. What are support vectors?

SVM finds the hyperplane that maximally separates two classes by maximising the margin, which is the perpendicular distance from the hyperplane to the nearest data points on each side. The decision boundary is not just any separating line but the one with the widest possible gap between the classes. Support vectors are the training samples that lie exactly on the margin boundary. They are the only points that define the hyperplane; all other training points are irrelevant. This gives SVM a key property: the decision boundary is determined by just a few critical examples rather than the entire dataset. The C parameter controls the trade-off between maximising the margin and allowing misclassification of training points. The kernel trick allows SVM to find non-linear boundaries by implicitly mapping data to higher dimensions.

Q — When would you choose SVM over Random Forest or XGBoost?

I would choose SVM over tree-based ensembles in three scenarios. First, when the dataset is small, say fewer than 10,000 samples, because SVM performs well with limited data and its computational cost is manageable. Second, when the data is high-dimensional with fewer samples than features, such as in genomics or text classification, because SVM with an

> appropriate kernel can still find effective boundaries while tree methods struggle. Third, when there is a clear margin of separation in the data, meaning the classes are relatively well-separated, because this is exactly the condition where SVM is most theoretically powerful. I would not choose SVM when the dataset is very large because training time scales poorly, or when I need probability outputs that are well-calibrated, or when I need interpretable feature importances.

SECTION 08 Words to Know

Term	Definition
KNN	K-Nearest Neighbours. No training; predicts by majority vote of K closest points.
n_neighbors (K)	The K in KNN. How many neighbours to consider. Key hyperparameter.
Euclidean distance	sqrt(sum((x1-x2)^2)). Default distance metric in KNN.
Curse of dimensionality	In high dimensions, all points become equally far apart. Distance loses meaning.
Naive Bayes	Probabilistic classifier using Bayes' Theorem with independence assumption.
GaussianNB	Naive Bayes for continuous numeric features (assumes Gaussian distribution).
MultinomialNB	Naive Bayes for count data (word counts). Standard for text classification.
BernoulliNB	Naive Bayes for binary features (word present or absent).
Laplace smoothing	alpha in NB. Prevents zero-probability for unseen words.
CountVectorizer	Converts text to word count matrix. Input for MultinomialNB.
TfidfVectorizer	Like CountVectorizer but weights rare words higher than common words.
SVM	Support Vector Machine. Finds maximum-margin hyperplane between classes.
Hyperplane	A flat subspace that separates classes. A line in 2D, plane in 3D.
Margin	The distance between the hyperplane and the nearest points of each class.
Support vectors	The training points that lie on the margin boundary. Define the hyperplane.
Kernel	Function that maps data to higher dimensions. 'rbf', 'linear', 'poly'.
RBF kernel	Radial Basis Function. Most common kernel. Good for non-linear problems.
C parameter (SVM)	Regularisation strength. High C = narrow margin, fewer train mistakes.
gamma parameter	Influence radius per point. High = complex boundary, low = smooth.
Pipeline	sklearn utility: chain scaler + model into one object with fit/predict.

SECTION HW Homework — Submit Before Day 21

> Create day20_homework.py. Always use Pipeline when your model needs scaling (KNN, SVM, Logistic Regression). Always compare all models against each other. This is the last single-algorithm homework — Day 21 starts feature engineering. Push to GitHub before midnight.

Exercise 1 — KNN K-selection (Beginner)

Using the Iris dataset, train KNN with k = 1, 3, 5, 7, 9, 11, 15, 21. For each k: record training and test accuracy. Plot both curves on one chart. Use 5-fold CV to find the best k. Create a Pipeline that includes StandardScaler so scaling is applied correctly. Write a comment explaining why k=1 gives 100% training accuracy.

Exercise 2 — Naive Bayes Spam Filter (Beginner-Intermediate)

Expand the spam filter from Section 02. Create a dataset of at least 30 emails (15 spam, 15 not spam). Use TfidfVectorizer instead of CountVectorizer. Train MultinomialNB. Print precision, recall, and F1 for both classes. Then test it on 5 new emails you write yourself and print the predicted class and probability. Write a comment: which words did the model learn are most associated with spam?

Exercise 3 — SVM C and gamma Grid Search (Intermediate)

Using the breast cancer dataset (scaled), run GridSearchCV over C = [0.01, 0.1, 1, 10, 100] and gamma = ['scale', 0.001, 0.01, 0.1, 1]. Try both 'rbf' and 'linear' kernels. Print a table showing CV accuracy for every combination. Report the best C, gamma, kernel combination. Plot the CV scores as a heatmap (C on x-axis, gamma on y-axis) for the RBF kernel.

Exercise 4 — Full Six-Algorithm Tournament (Intermediate)

Run the full six-algorithm comparison (KNN, Naive Bayes, SVM, Random Forest, XGBoost, LightGBM) on TWO datasets of your choice from sklearn.datasets (not breast cancer or wine — try load_digits, load_iris, fetch_california_housing, or make_classification). For each dataset report: 5-fold CV accuracy, test accuracy, and training time. Create a bar chart comparing test accuracy for all models on each dataset. Identify patterns: which model wins on which type of data?

Exercise 5 — Pipeline Best Practice (Intermediate)

Build a clean Pipeline for each of the three models today: (1) Pipeline([StandardScaler, KNN]), (2) Pipeline([StandardScaler, SVM_RBF]), (3) GaussianNB (no scaling needed). Run cross_val_score on each Pipeline. Demonstrate that training WITHOUT the Pipeline (scaling train and test separately) gives the same result, but using Pipeline is safer in production. Write a comment explaining the data leakage risk when scaling outside a Pipeline.

Stretch Challenge — Tier A students

Implement KNN from scratch using only NumPy. Your class should have: __init__(k), fit(X, y) that just stores X and y, predict(X_test) that for each test point computes Euclidean distance to all training points, finds the k smallest distances, takes majority vote, returns predicted class. Test on Iris and verify accuracy is within 1% of sklearn's KNeighborsClassifier. Then add a predict_proba method that returns the fraction of k neighbours belonging to each class. Verify it matches sklearn's predict_proba.